Abstract class

→it is considered as a blue print for other classes.
→A class contains one or more abstract methods is called an abstract class.
→we cannot create an object to abstract class.
→it provides the default functionality of the base class.
→it defines a common API for a set of sub classes, useful where a third party is providing plugings in an application.

Syntax:

```
from abc import ABC#here ABC is a metaclass
```

Abstract method:

it is a method that has a declaration but does not have any implementation.
(or)
A method without body is called abstract method.

Syntax:

```
from abc import ABC,abstractmethod
class class_name(ABC):
    @abstractmethod
    def method_name(self):
        statements
```

Example:

```
def name(self):
    pass
```

concrete method (or) complete method:

A method with body is called concrete method.

Example:

```
def name(self):
    print(“welcome india”)
```

Note:
abstract class contains abstract methods and complete methods

Example:

```
from abc import ABC,abstractmethod
class shape(ABC):#abstract class
  def common(self):
    print("this is complete method")
  @abstractmethod#decorator
  def area(self):
    pass
```

```
    @abstractmethod
    def perimeter(self):
        pass
class square(shape):#sub class
    def __init__(self,side):
        self.side=side
    def area(self):
        return self.side*self.side
    def perimeter(self):
        return 4*self.side
class Rectangle(shape):#another sub class for shape class
    def __init__(self,length,breath):
        self.length=length
        self.breath=breath
    def area(self):
        return self.length*self.breath
    def perimeter(self):
        return 2*(self.length+self.breath)
s1=square(4)
s1.common()
print(s1.area())
print(s1.perimeter())
r=Rectangle(23,45)
r.common()
print(r.area())
print(r.perimeter())
```

INTERFACE

→this concept is not explicitly available in python.

→An interface is similar to class.

→an interface is an abstract class which contain only abstract methods but not a single concrete method.

Syntax:

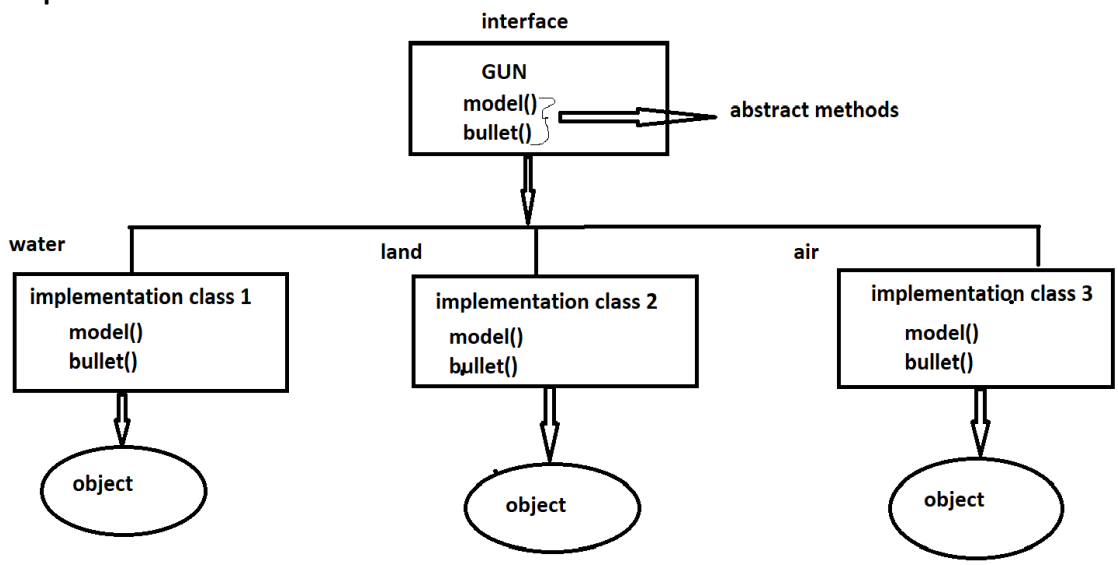
```
from abc import ABC,abstractmethod
class class_name(ABC):#interface
    @abstractmethod
    def function_name(self):
        pass
```

Rules:

- **All methods of an interface is abstract.**

- **we cannot create object to interface because interface contains abstract methods or incomplete methods.**
- **if a class is implementing it has to define all the methods declared in the interface, the class must be declared as abstract.**
- **if a class is implementing it has to define all the methods given in that interface.**
- **we use interface when all the features needed to be implemented differently for different objects.**

Example:



Example:

```
from abc import ABC,abstractmethod
class Father(ABC):
   @abstractmethod
   def disp1(self):
      pass
   @abstractmethod
   def disp2(self):
      pass
class  child(Father):
   def disp1(self):
      print("child class")
      print("display abstract method called")
   def disp2(self):
      print("grandchild class")
      print("display abstract method")
class Granchild(child):
   pass
gc=Granchild()
gc.disp1()
```

```
gc.disp2()
```